Universidad Autónoma de Entre Ríos
Facultad de Ciencia y Tecnología

Memoria Final

Trabajo Práctico Final Integrador
Patrón Proxy / Singleton / Observer en entorno AWS

Ingeniería de Software II

Alumno: Julián Olivera

Fecha: 2 de noviembre de 2026

Versión 1.0.0

Repositorio: https://github.com/julianAO2002/IS2_TPFI_OLIVERA

Índice

1. Resumen
2. Problema
3. Diseño
4. Implementación
5. Pasos realizados
6. Validación y verificación
7. Calidad del código
8. Integración y entrega continua
9. Cambios y defectos encontrados
10. Limitaciones conocidas
11. Uso de inteligencia artificial
12. Conclusiones
13. Anexo: ejecución del sistema

1. Resumen

Se desarrollaron tres programas en Python que permiten consultar y modificar datos corporativos alojados en AWS DynamoDB. Un servidor concentra el acceso a las tablas, deja una pista de auditoría de cada acción y avisa los cambios a los clientes suscriptos.

Indicador	Resultado
Pruebas automatizadas	197 aprobadas de 197: 165 unitarias y 32 de aceptación
Aceptación contra AWS real	32 de 32 aprobadas, en 68 segundos
Cobertura de código	96.80% (mínimo exigido: 85%)
ruff, black, PEP8, PEP257, mypy, pyright, bandit	Sin errores ni observaciones
TruffleHog sobre el historial	0 secretos
Workflow de CI	5 de 5 controles aprobados en cada PR, en Ubuntu y Windows

Cumplimiento de la consigna

Requisito del enunciado	Resultado	Evidencia
SingletonClient: get, set, list; salida a archivo o pantalla	Cumplido	Figuras 7 a 10; `src/tpfi/clients/singleton_client.py`
ObserverClient: suscripción, múltiples avisos, reintento	Cumplido	Figuras 6, 15 y 16
Servidor con Proxy, Singleton y Observer; pista auditable	Cumplido	Figuras 5 y 17; sección 3
Tablas accedidas con sendos Singleton	Cumplido	`src/tpfi/persistence/`; pruebas de patrón
Suscripciones con Observer; actualizaciones con Proxy	Cumplido	`src/tpfi/server/observer.py` y `proxy.py`
Sesión con uuid4, CPU con getnode y platform	Cumplido	Figura 17; `corporate_log.py`
Debug con logging por salida estándar, activado con -v	Cumplido	Figura 5
Aceptación automatizada de los 5 grupos de casos	Cumplido	Figura 20: 32 casos sobre AWS real
UML de actividad y de estado	Cumplido	Figuras 1 y 2
Pruebas que verifican los patrones	Cumplido	Sección 6.2
venv, CookieCutter, CONTEXT, README, CHANGELOG, requirements, MIT, VERSION, BUILD	Cumplido	Raíz del repositorio; sección 5
ruff, black, PEP8, PEP257, MyPy, PyRight sin errores	Cumplido	Figura 18
Pytest con cobertura de 85% o mejor	Cumplido: 96.80%	Figura 19
bandit sin observaciones	Cumplido	Figura 18
Trufflehog (opcional)	Cumplido	Figura 21 y job de CI
Workflow de CI/CD en GitHub	Cumplido	Figuras 24 a 27
Documentación con pdoc actualizada con cada PR	Cumplido	Figuras 28 y 29

2. Problema

Un sistema aplicativo necesita servicios corporativos centralizados: dirección oficial, CUIT, teléfono de contacto e identificador único de secuencia. Se implementan tres funciones (recuperar, modificar y listar) sobre dos tablas de DynamoDB que ya existen: `CorporateData`, con los datos, y `CorporateLog`, con los registros de acceso y modificación.

Puntos del enunciado que requirieron una decisión

Situación	Decisión
La Ilustración 1 muestra GET para subscribe, pero el texto dice POST tres veces.	Se sigue el texto: todos los pedidos usan POST.
La nota de aplicación indica la región us-west-2.	Las tablas y las credenciales están en us-east-1. La región se toma del perfil de AWS, no del código.
La Ilustración 1 incluye una respuesta OK||Error del observador al servidor; el texto no la describe.	No se implementa ese mensaje. El servidor detecta la caída de un suscripto por el fallo de envío o el cierre del socket.
El enunciado nombra REQUIREMENTS.TXT y requirements.txt.	Se usa `requirements.txt`, el nombre que entiende pip por convención.
CorporateData es compartida por todos los alumnos.	Las pruebas reales solo escriben claves `OLIVERA-TEST-` y solo borran esas claves.

3. Diseño

Arquitectura

Los clientes se conectan al servidor por TCP. Cada conexión se atiende en un hilo propio y cada pedido pasa por el proxy, que audita y recién después delega en el servicio real.

```
Clientes --TCP (JSON con POST)--> Servidor --> Proxy --> Servicio real --> CorporateData
                                                |                          (Singleton)
                                                |--> CorporateLog (Singleton)
                                                '--> Suscripciones (Observer) --> Clientes
```

Patrones

Patrón	Problema que resuelve	Implementación
Singleton	Varios hilos necesitan acceder a las tablas; debe haber un solo punto de acceso por tabla.	Metaclase `SingletonMeta` con un lock; la usan `CorporateDataTable` y `CorporateLogTable`.
Proxy	Toda operación debe auditarse, y los cambios deben notificarse, sin mezclar esas reglas con el acceso a datos.	`CorporateServiceProxy` implementa la misma interfaz que `RealCorporateService`. Si la auditoría falla, la operación no se ejecuta.
Observer	Hay que avisar a una cantidad variable de clientes sin que quien modifica datos los conozca.	`SubscriptionManager` es el sujeto y `SocketObserver` el observador concreto, con su propia cola de envío.

Decisiones técnicas

Decisión	Motivo
Mensajes con cabecera `Content-Length`	TCP entrega un flujo de bytes: sin esa cabecera el receptor no sabe dónde termina cada mensaje.
Escrituras condicionales en DynamoDB	Leer y luego escribir permitiría que dos clientes se pisen. La condición se evalúa de forma atómica.
Lock dentro de cada tabla	Los recursos de boto3 no son seguros entre hilos y la instancia Singleton es compartida.
Auditar antes de operar	Así no puede existir un cambio sin registro. Un intento fallido queda registrado igual.
`socket.create_server`	Resuelve la diferencia entre sistemas: en Windows, SO_REUSEADDR permitiría dos servidores en el mismo puerto.
Validación en cliente y en servidor	El cliente falla antes de conectarse, y el servidor nunca confía en lo que recibe por la red.

Diagrama de actividad: list, get y set

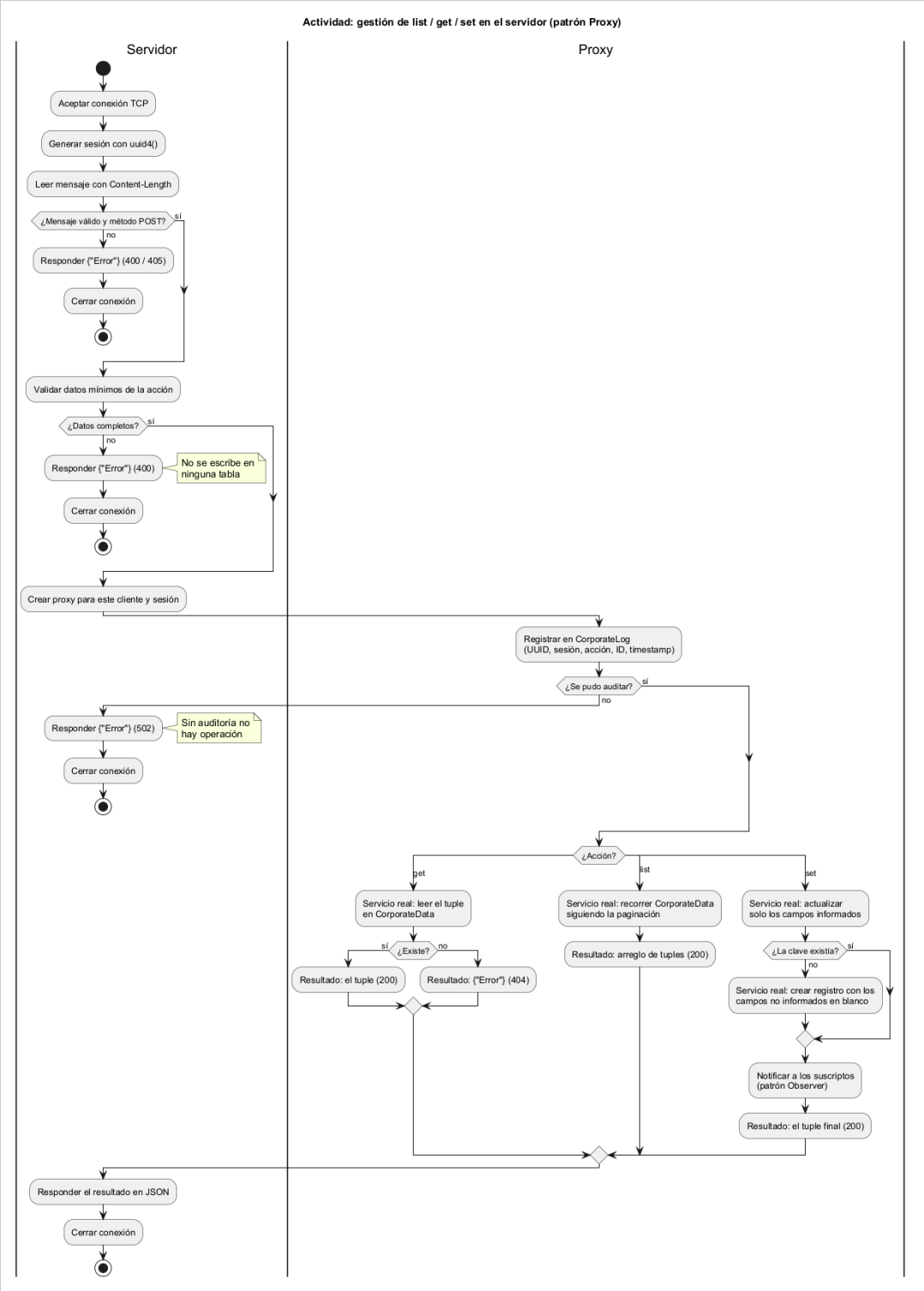


Figura 1. Actividad de la gestión de list, get y set en el servidor (patrón Proxy).

Diagrama de estado: suscripción

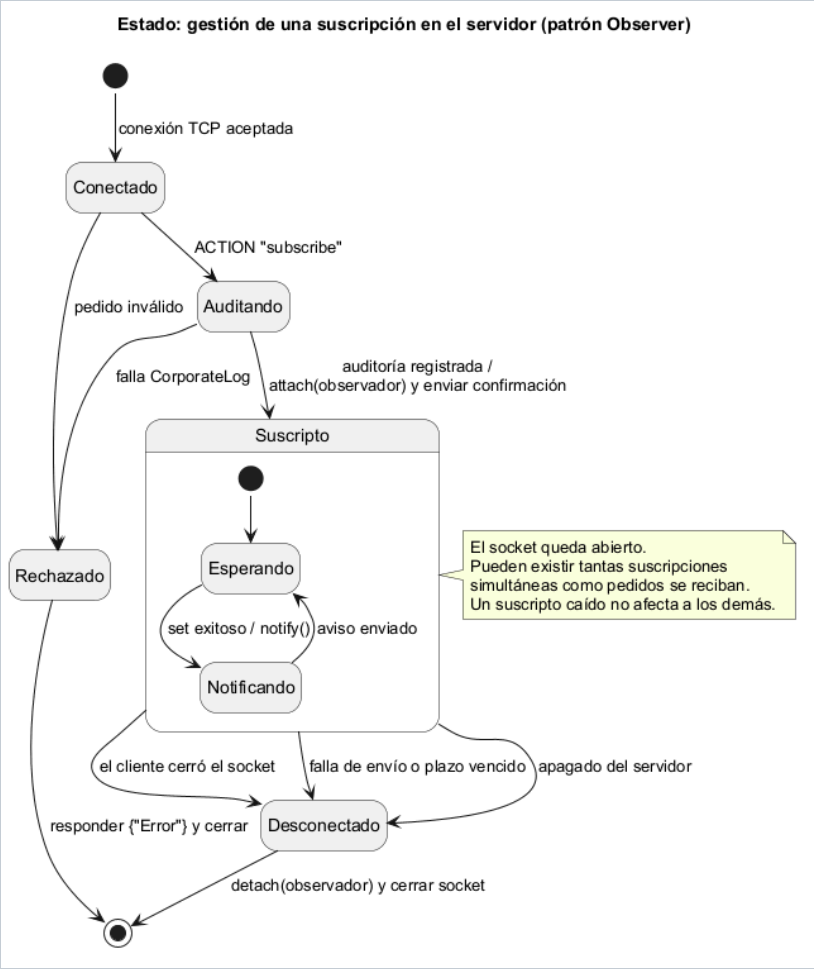


Figura 2. Estado de una suscripción en el servidor (patrón Observer).

Diagrama de clases

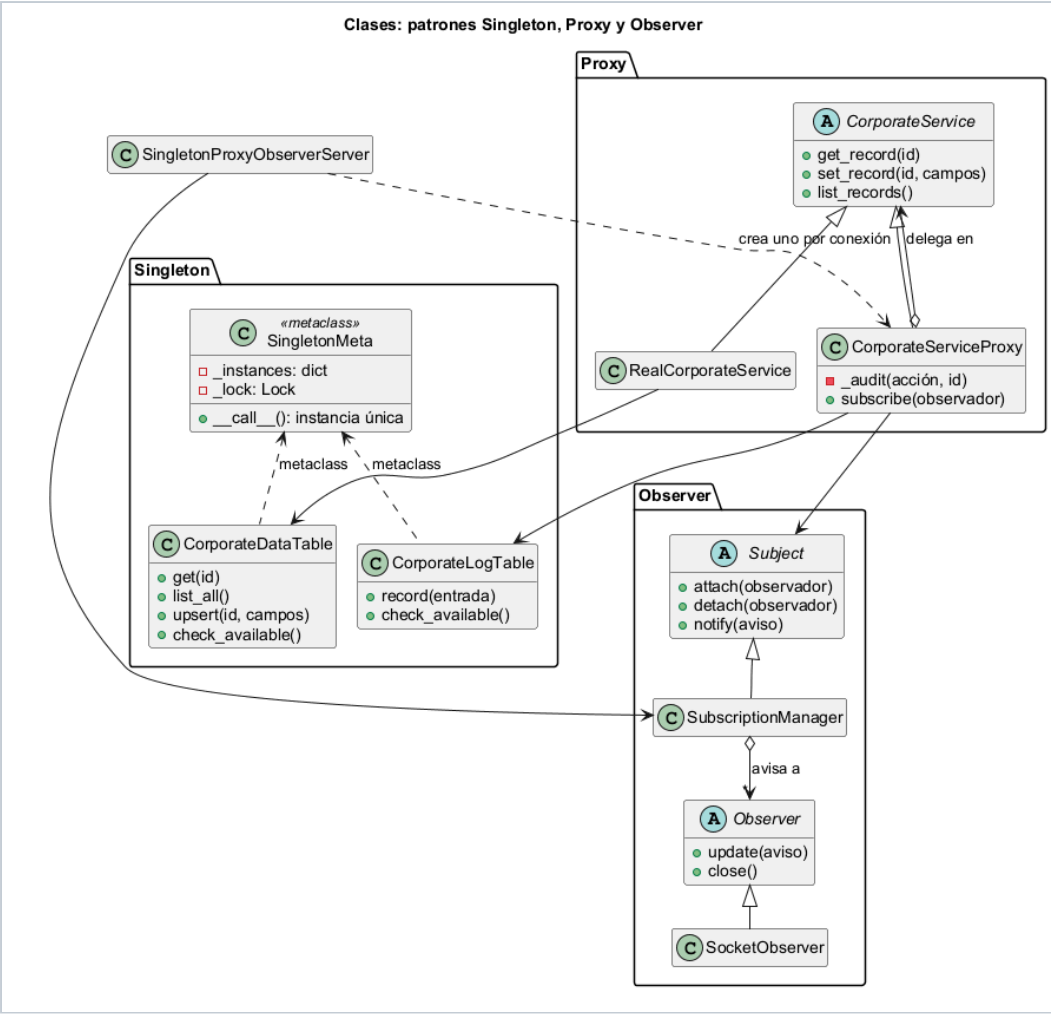


Figura 3. Clases que implementan los tres patrones.

4. Implementación

```
singletonproxyobserver.py    Lanzador del servidor
singletonclient.py           Lanzador del cliente de consultas
observerclient.py            Lanzador del cliente suscriptor
src/tpfi/common/             Protocolo, errores, línea de comandos y versión
src/tpfi/persistence/        Singleton y acceso a CorporateData y CorporateLog
src/tpfi/server/             Proxy, Observer, servicio real y servidor TCP
src/tpfi/clients/            SingletonClient y ObserverClient
tests/unit/                  Pruebas unitarias y de patrones
tests/acceptance/            Pruebas de aceptación con procesos reales
docs/                        UML, evidencias y memoria
```

Todo el código está en clases, salvo los lanzadores y las funciones de utilidad. Los comentarios y docstrings están en español y explican el porqué de cada decisión. Los identificadores están en inglés, para coincidir con los nombres de los patrones y del enunciado.

Formato de los mensajes

```
POST /tpfi HTTP/1.1
Host: localhost
Content-Type: application/json
Content-Length: 67

{"UUID": "39908810720395", "ID": "UADER-FCyT-IS2", "ACTION": "get"}
```

Las respuestas exitosas devuelven el tuple o el arreglo de tuples. Los errores devuelven `{"Error": "descripción"}`. Cada suscriptor recibe `{"UUID", "ID", "ACTION": "change", "DATA"}`, según la Ilustración 1.

5. Pasos realizados

1. **Análisis.** Lectura del enunciado, la nota de aplicación y los programas de diagnóstico. Consulta del esquema real de las tablas: clave `id` de tipo texto, y capacidad de lectura de 1 RCU en CorporateData y 10 en CorporateLog.
2. **Esqueleto.** Generación con cookiecutter-pypackage (primer commit, sin modificar).
3. **Ambiente.** Creación del venv e instalación de versiones fijas con `requirements.txt`.
4. **Herramientas.** Configuración de ruff, black, pycodestyle, pydocstyle, mypy, pyright, bandit, pytest y los workflows de CI y CD.
5. **Protocolo y Singleton.** Desarrollo guiado por pruebas: primero la prueba que falla y después el código.
6. **Proxy, Observer y servidor.** Mismo método.
7. **Clientes.** SingletonClient y ObserverClient, con reconexión.
8. **Aceptación.** Primero contra un DynamoDB simulado (moto) y después contra AWS real.
9. **Revisión.** Revisión independiente por lentes de riesgo, resiliencia y fiabilidad.
10. **Publicación.** Repositorio en GitHub, con búsqueda de secretos previa al primer push.
11. **Integración.** PR 1, PR 6 y PR 7 integrados con todos los controles aprobados.
12. **Evidencias.** Corridas reales y capturas sobre el código final.
13. **Memoria.** Este documento, generado a partir de las salidas guardadas.

Commits principales

Commit	Descripción
0716354	Esqueleto generado con cookiecutter-pypackage
7e4feb5	Herramientas de calidad, CI y CD
40b751e	Protocolo y acceso Singleton a DynamoDB
ba40c72	Servidor Proxy y Observer, clientes y pruebas de aceptación
ffccc45	La auditoría se lee por clave, para no agotar la capacidad de CorporateLog
8956a17	Cola de envío por suscriptor, para no frenar los set
cca859d	Registro de build 2, cambios y prueba de versión
bb852c9	Documentación de todos los módulos y verificación de las páginas generadas
c90219f	Exportación del paquete y pruebas de la generación de documentación

6. Validación y verificación

> Salvo aclaración, las capturas son de ventanas reales de terminal, ejecutando los programas contra las tablas reales de AWS. Las salidas completas están en `docs/evidencias/salidas` y los logs de debug de las pruebas en `docs/evidencias/logs`. El UUID que se ve es el número de CPU obtenido con `uuid.getnode()`.

6.1 Casos de prueba de aceptación

Grupo del enunciado	Casos	Qué se verifica
Camino feliz de cada acción	7	get, set, list y subscribe; archivo de salida; registro en CorporateLog
Argumentos malformados (cada programa)	10	Terminan con código 2 y mensaje de error
Sin datos mínimos	11	Se rechazan en cliente y servidor; las tablas no cambian
Servidor caído	3	Error en el cliente; el observador reintenta y se vuelve a suscribir
Levantar dos veces el servidor	1	El segundo falla y el primero sigue atendiendo

Cada prueba lee las tablas directamente con boto3, sin pasar por el sistema bajo prueba. Por eso el resultado no depende de lo que el sistema dice haber hecho.

6.2 Pruebas de los patrones

Patrón	Qué demuestran las pruebas
Singleton	Dos construcciones devuelven el mismo objeto; cada tabla tiene su instancia; 16 hilos simultáneos reciben la misma; el constructor corre una sola vez.
Proxy	Comparte la interfaz con el servicio real y delega en él; audita antes de operar; sin auditoría no toca el servicio real; solo un set exitoso notifica.
Observer	Notifica a todos; admite varias suscripciones del mismo UUID; un suscripto caído o trabado no afecta a los demás; los avisos llegan en orden.

6.3 Diagnóstico del ambiente

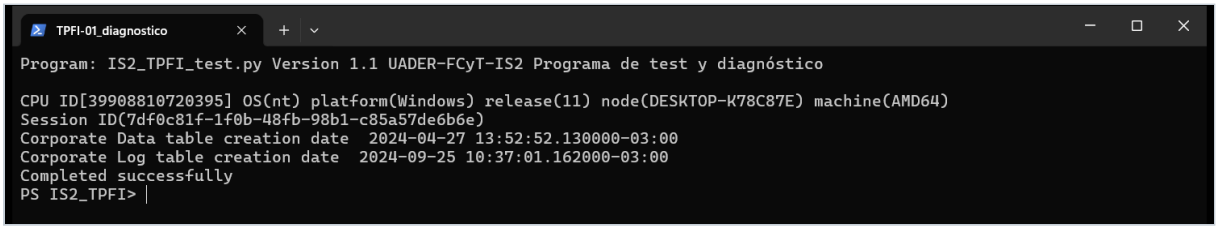


Figura 4. Condición: acceso a las tablas verificado con el programa IS2_TPFI_test.py de la cátedra.

6.4 Camino feliz

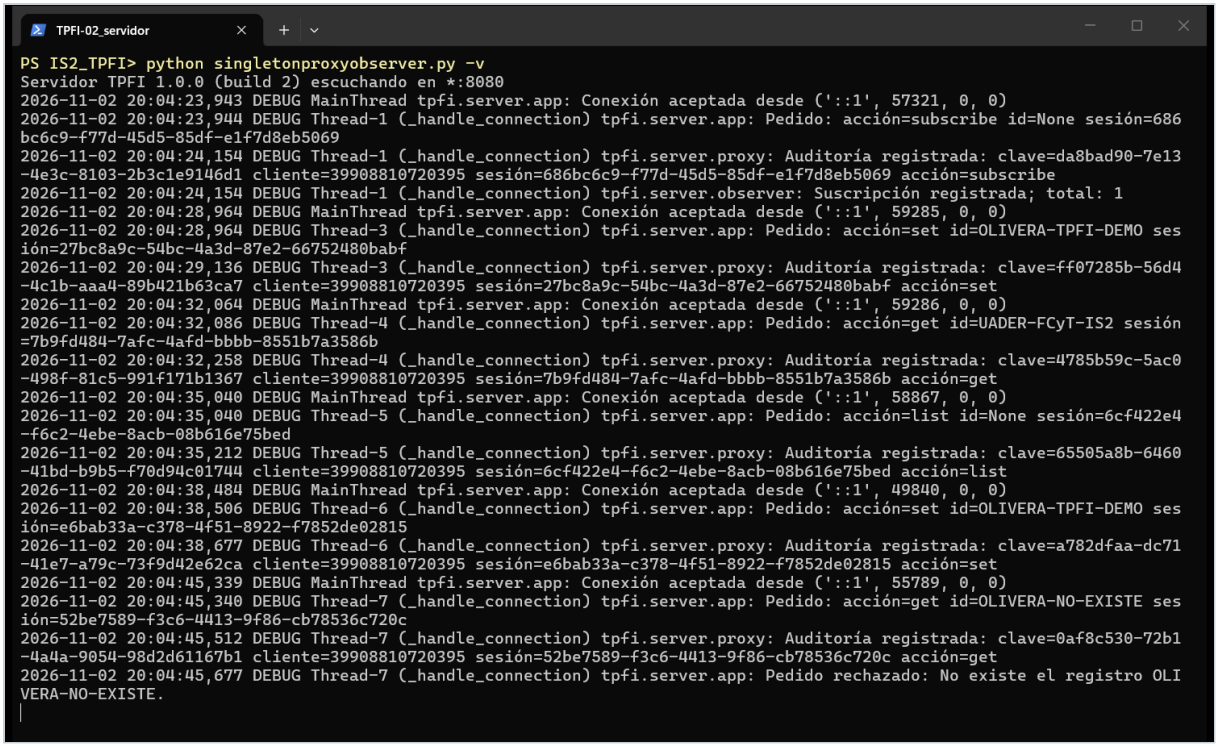


Figura 5. Condición: traza del servidor con -v. Cada acción muestra su sesión uuid4 y su clave de auditoría.

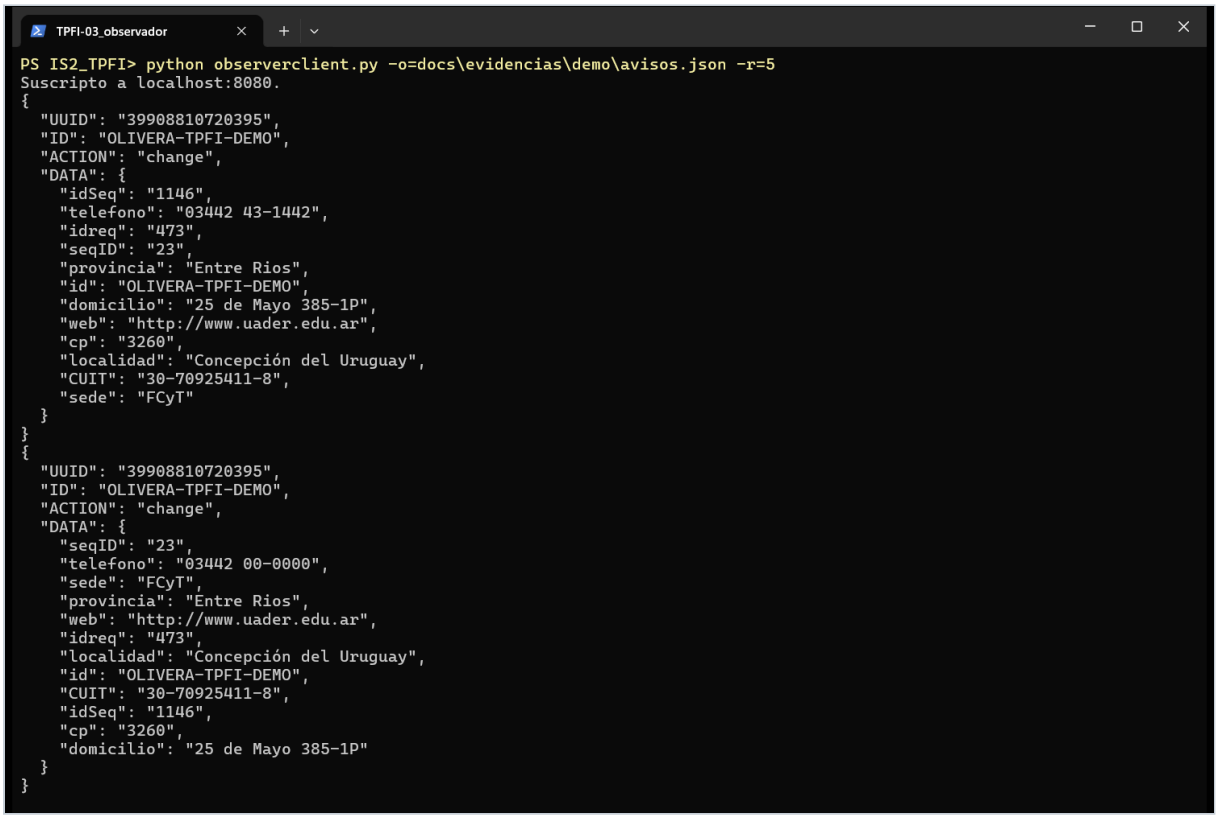


Figura 6. Condición: el observador recibe cada cambio (subscribe) y mantiene abierto el socket.

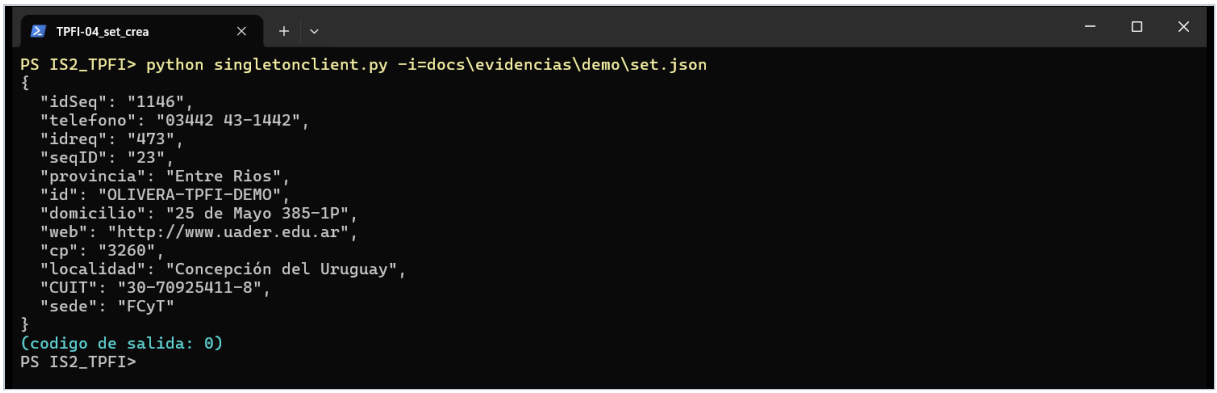


Figura 7. Condición: set de una clave nueva, con todos los campos del tuple.

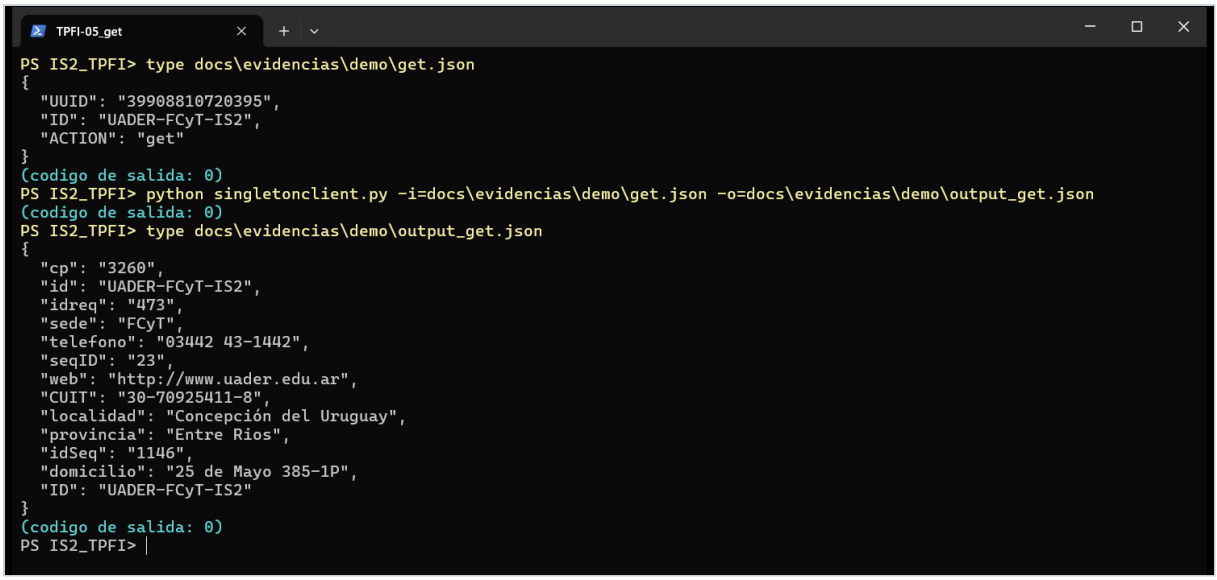


Figura 8. Condición: get con el pedido de entrada y la salida grabada en output_get.json.

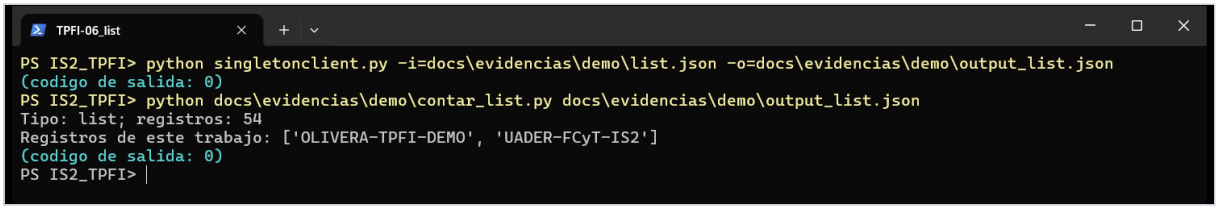


Figura 9. Condición: list devuelve un arreglo con todos los tuples de la tabla.

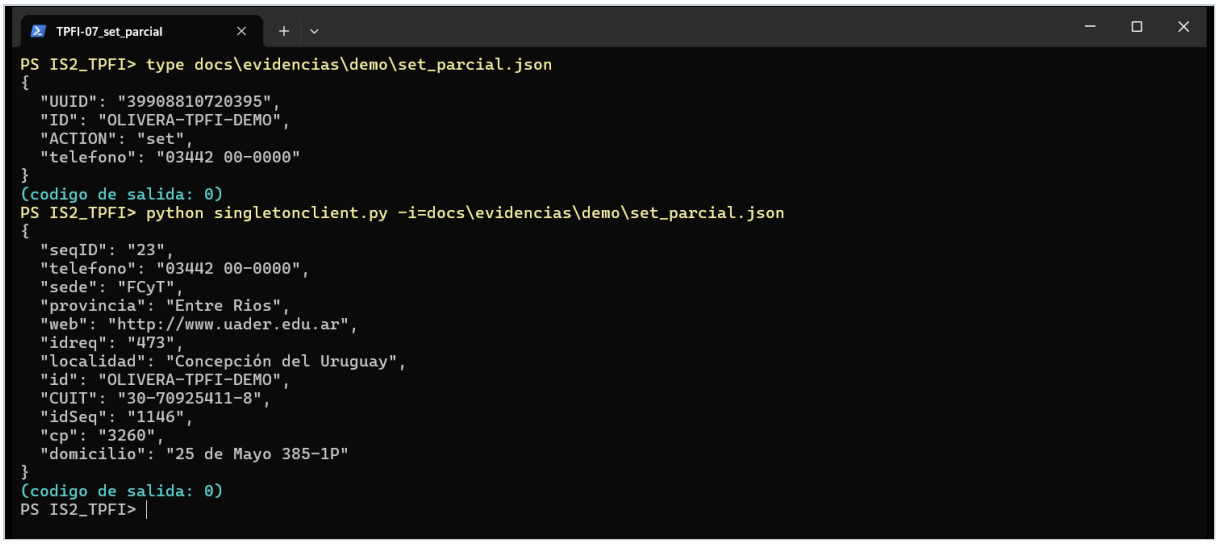


Figura 10. Condición: set de un solo campo; los demás no se modifican.

6.5 Casos de error

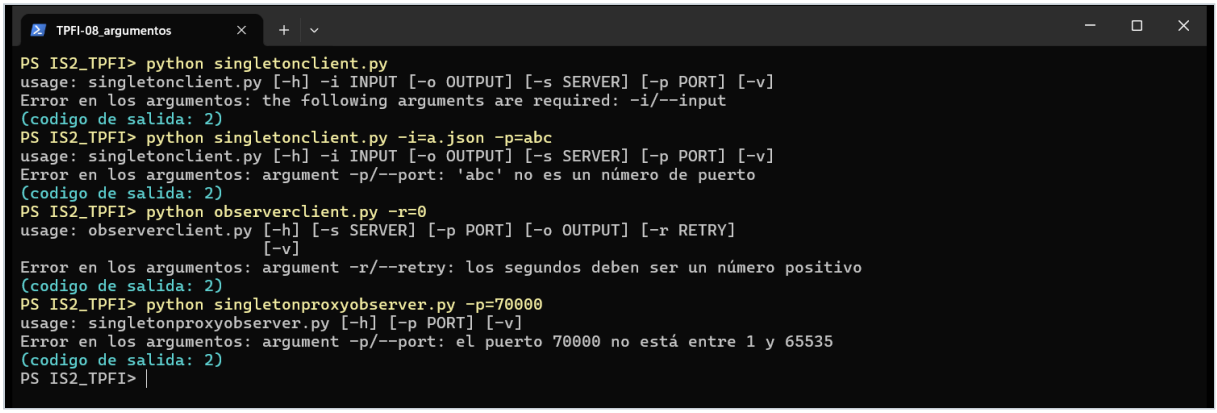


Figura 11. Condición: argumentos malformados en cada programa (código de salida 2).

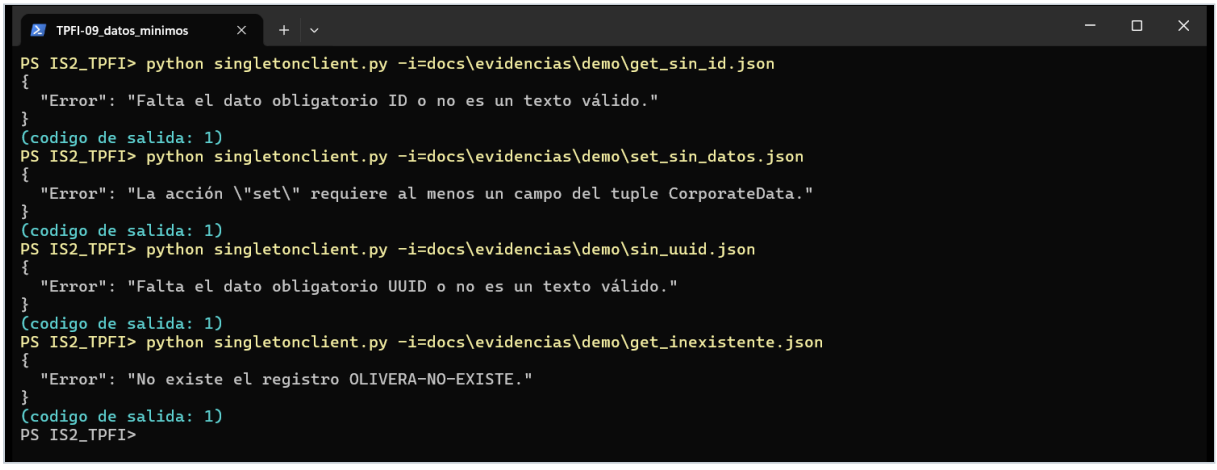


Figura 12. Condición: pedidos sin datos mínimos y registro inexistente.



Figura 13. Condición: intento de levantar dos veces el servidor.

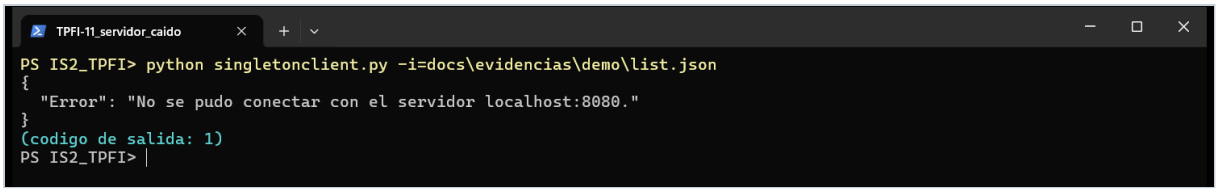


Figura 14. Condición: SingletonClient con el servidor caído.

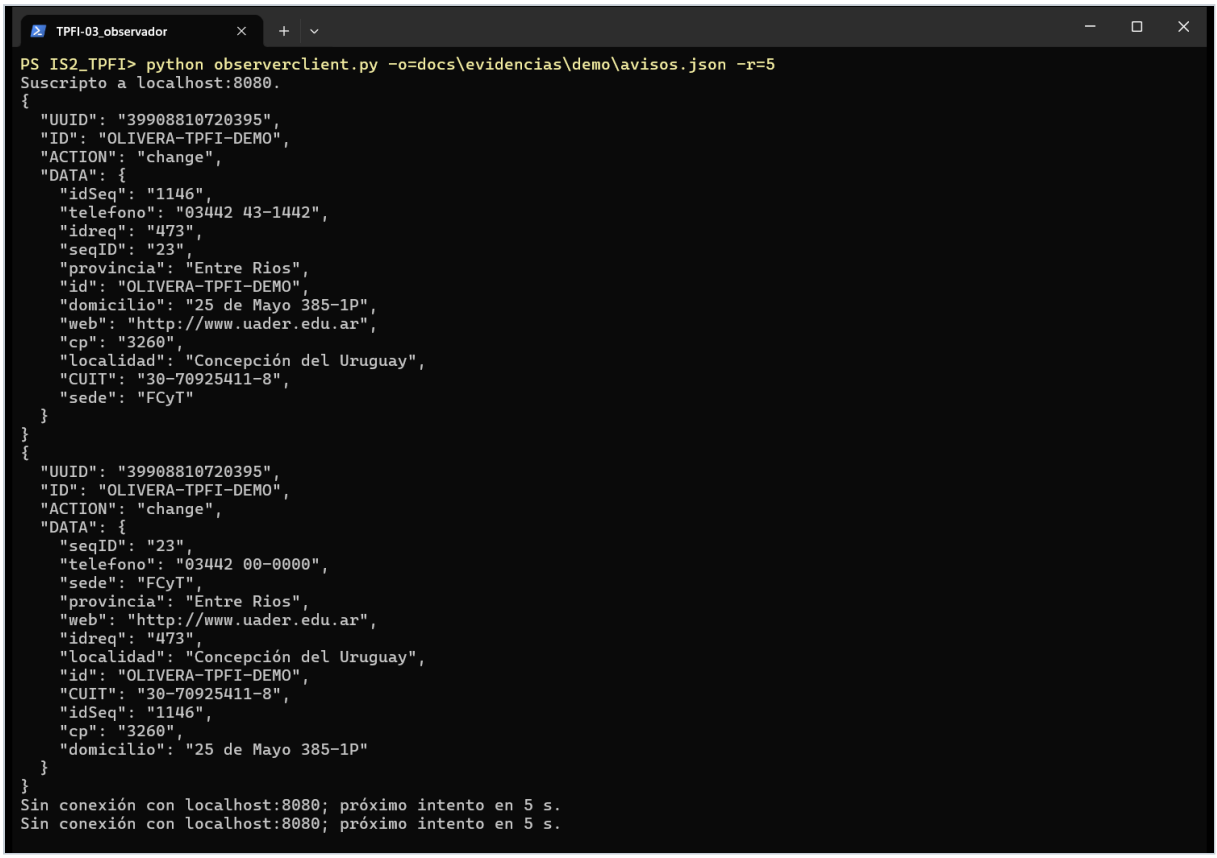


Figura 15. Condición: ObserverClient reintenta con el servidor caído (-r=5).

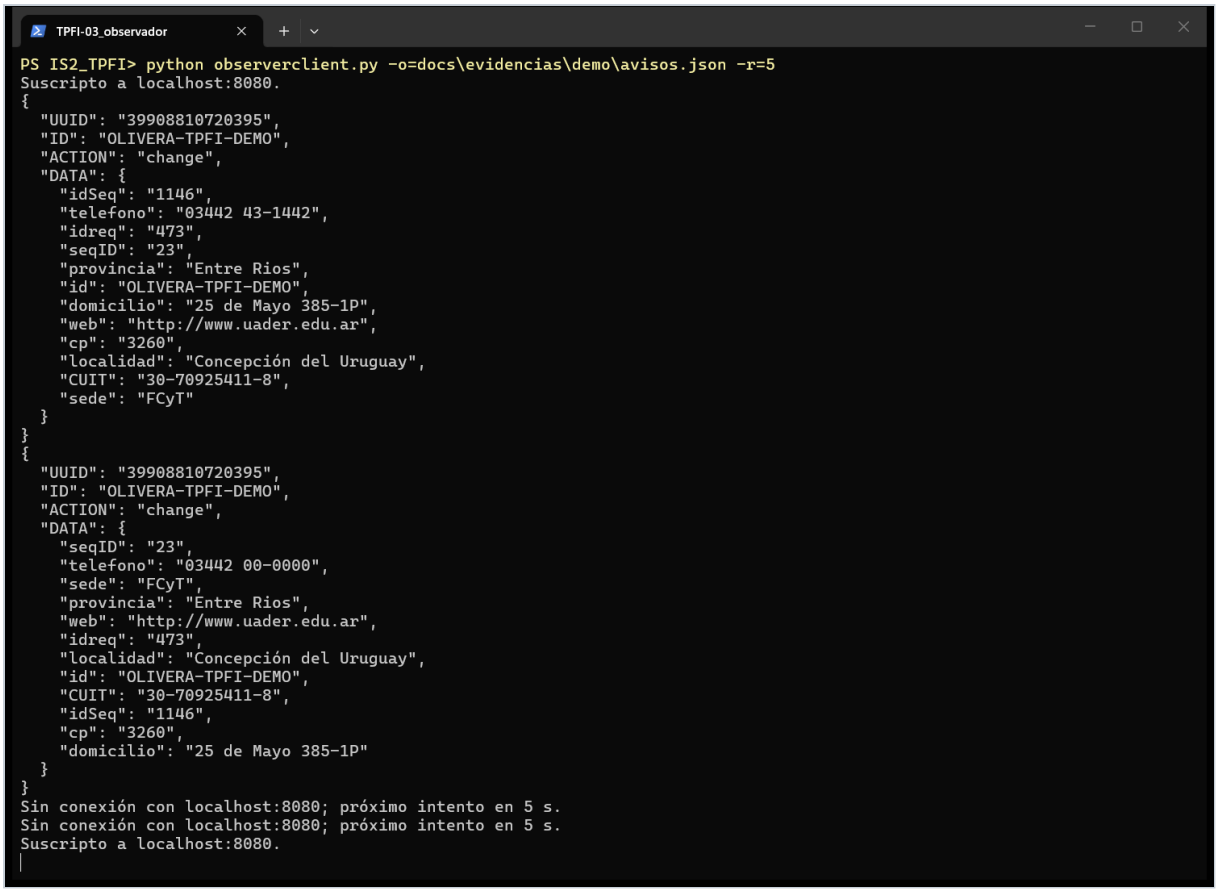


Figura 16. Condición: al volver el servidor, ObserverClient se suscribe de nuevo solo.

6.6 Impacto en las tablas

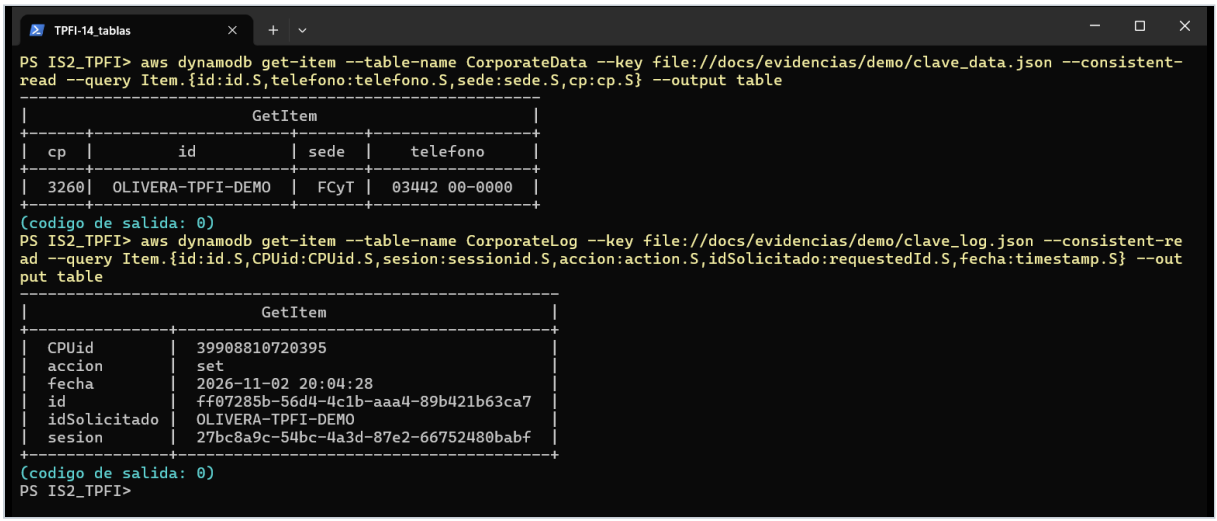


Figura 17. Condición: lectura directa con AWS CLI de CorporateData y de la fila de auditoría del set.

7. Calidad del código

Herramienta	Control	Resultado
ruff	Reglas de formato y errores frecuentes	Sin errores
black	Formato consistente	Sin cambios pendientes
pycodestyle	PEP8	Sin errores
pydocstyle	PEP257 en docstrings	Sin errores
mypy	Tipos, en modo estricto	Sin errores
pyright	Tipos	0 errores, 0 advertencias
bandit	Seguridad básica	Sin observaciones
pytest	Pruebas y cobertura	197 aprobadas, 96.80%
TruffleHog 3.99.8	Secretos en el historial	0 verificados, 0 sin verificar

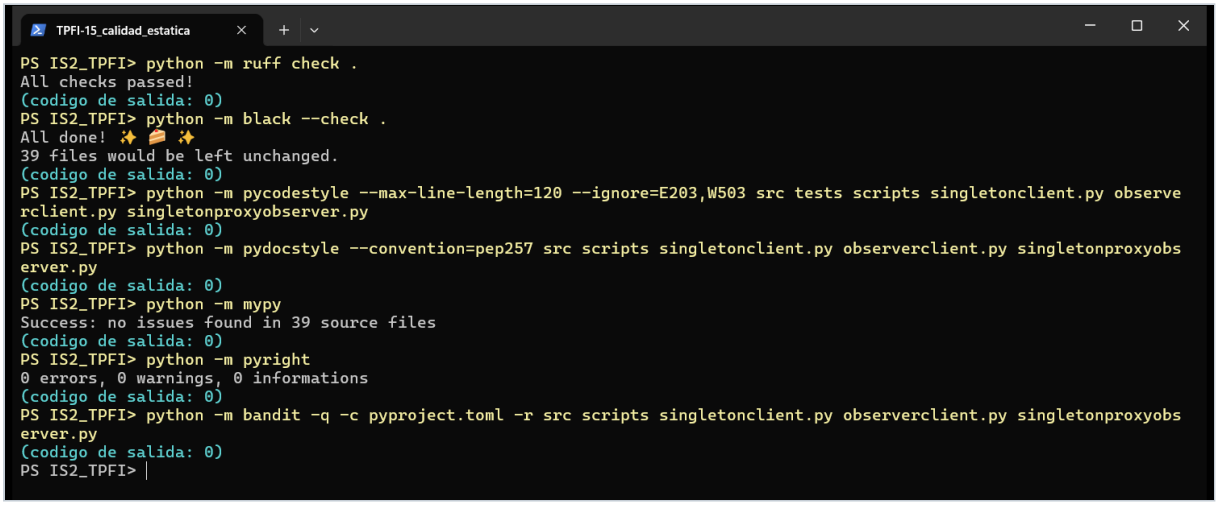


Figura 18. Condición: ruff, black, PEP8, PEP257, mypy, pyright y bandit terminan con código 0.

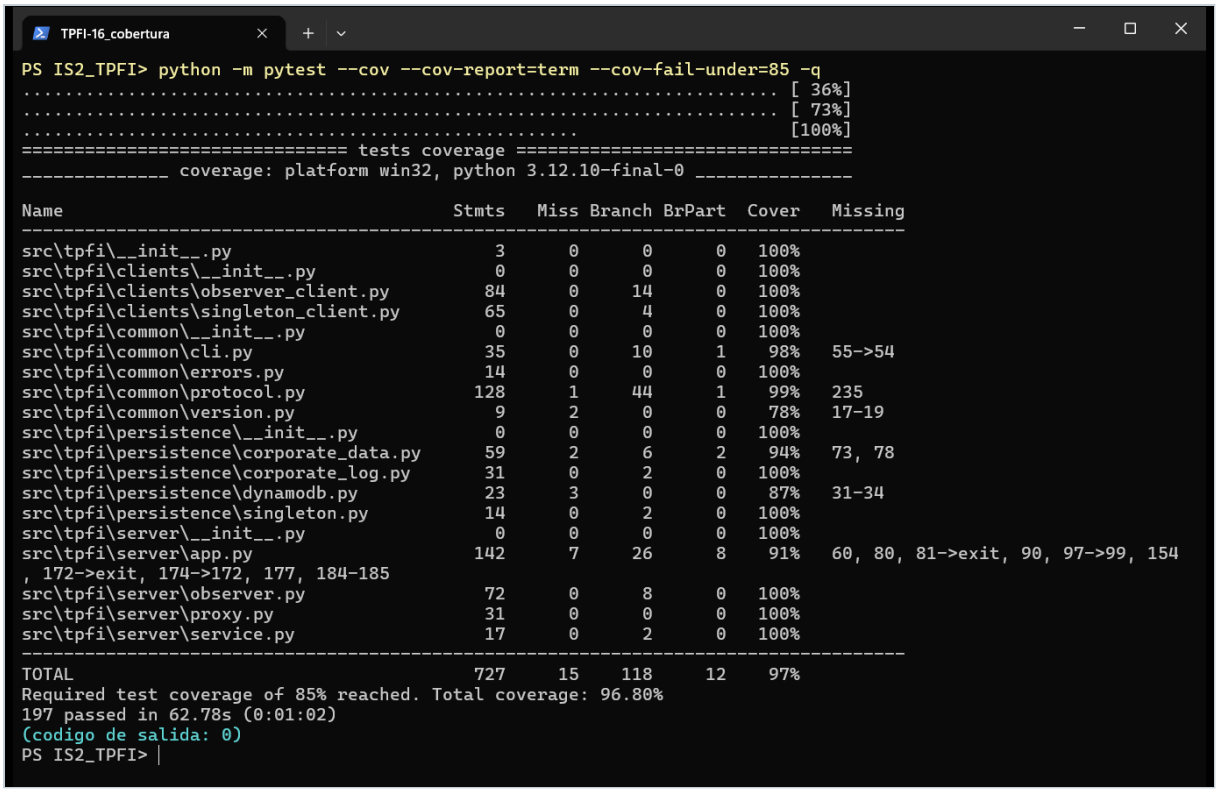


Figura 19. Condición: 197 pruebas aprobadas y cobertura de 96.80%.

Cobertura por módulo

```
Name                                    Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------------------------------
src\tpfi\__init__.py                        3      0      0      0   100%
src\tpfi\clients\__init__.py                0      0      0      0   100%
src\tpfi\clients\observer_client.py        84      0     14      0   100%
src\tpfi\clients\singleton_client.py       65      0      4      0   100%
src\tpfi\common\__init__.py                 0      0      0      0   100%
src\tpfi\common\cli.py                     35      0     10      1    98%   55->54
src\tpfi\common\errors.py                  14      0      0      0   100%
src\tpfi\common\protocol.py               128      1     44      1    99%   235
src\tpfi\common\version.py                  9      2      0      0    78%   17-19
src\tpfi\persistence\__init__.py            0      0      0      0   100%
src\tpfi\persistence\corporate_data.py     59      2      6      2    94%   73, 78
src\tpfi\persistence\corporate_log.py      31      0      2      0   100%
src\tpfi\persistence\dynamodb.py           23      3      0      0    87%   31-34
src\tpfi\persistence\singleton.py          14      0      2      0   100%
src\tpfi\server\__init__.py                 0      0      0      0   100%
src\tpfi\server\app.py                    142      7     26      8    91%   60, 80, 81->exit, 90,
97->99, 154, 172->exit, 174->172, 177, 184-185
src\tpfi\server\observer.py                72      0      8      0   100%
src\tpfi\server\proxy.py                   31      0      0      0   100%
src\tpfi\server\service.py                 17      0      2      0   100%
-----------------------------------------------------------------------------------
TOTAL                                     727     15    118     12    97%
```

Las líneas sin cubrir corresponden a fallos del entorno difíciles de provocar, como la falta de región de AWS o la ausencia de los archivos VERSION y BUILD.

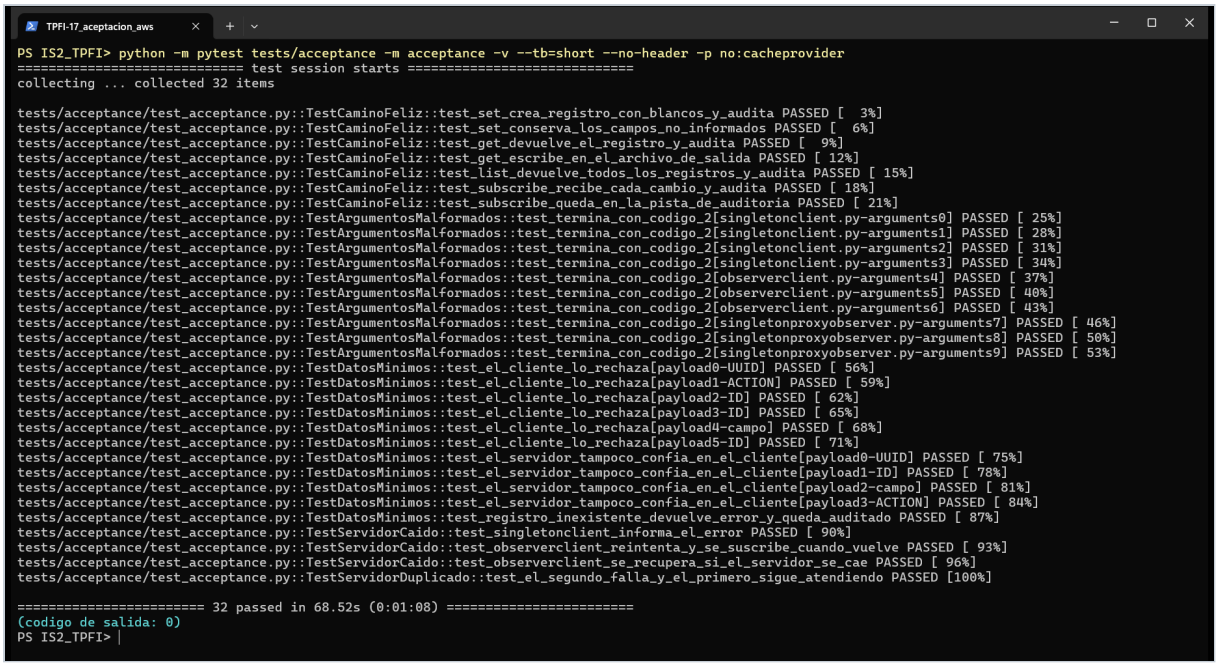


Figura 20. Condición: 32 pruebas de aceptación aprobadas contra las tablas reales de AWS.



Figura 21. Condición: TruffleHog sobre todo el historial de git, sin secretos.

Manejo de credenciales

- Las credenciales nunca se escriben en el código: boto3 las toma del perfil de AWS.
- El archivo de credenciales de la cátedra queda fuera del repositorio.
- `.gitignore` excluye `*credentials*.json` y `*.pem`.
- Además de TruffleHog, se buscó el valor literal de la clave en archivos e historial: 0 coincidencias.
- El CI no recibe credenciales de AWS: sus pruebas usan moto.

8. Integración y entrega continua

Workflow	Se dispara	Qué hace
CI (`ci.yml`)	En cada PR	Calidad estática; pruebas en Ubuntu y Windows con cobertura mínima de 85%; TruffleHog; documentación pdoc.
CD (`cd.yml`)	Al integrar en main	Repite el CI como compuerta; construye el paquete; publica la documentación en GitHub Pages.

Las acciones están fijadas por hash de commit, para que nadie pueda cambiar su contenido sin que se note. Las capturas de esta sección se tomaron sin iniciar sesión, desde el sitio público.

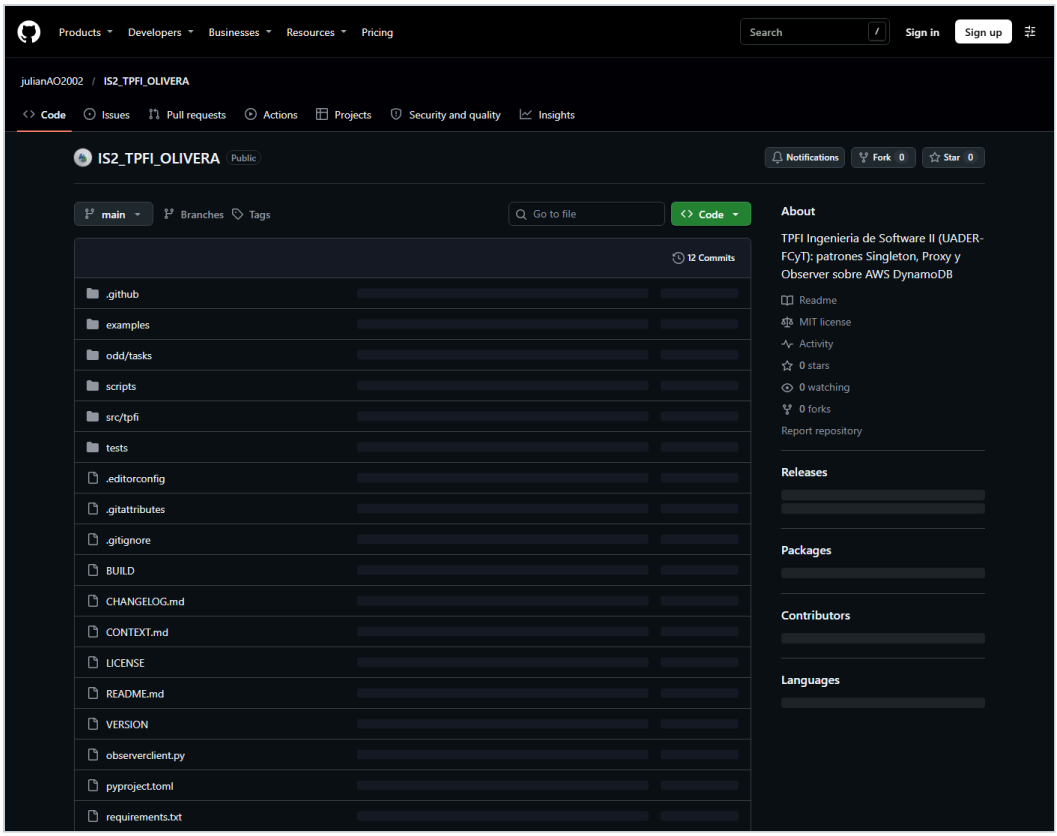


Figura 22. Condición: repositorio público en GitHub.

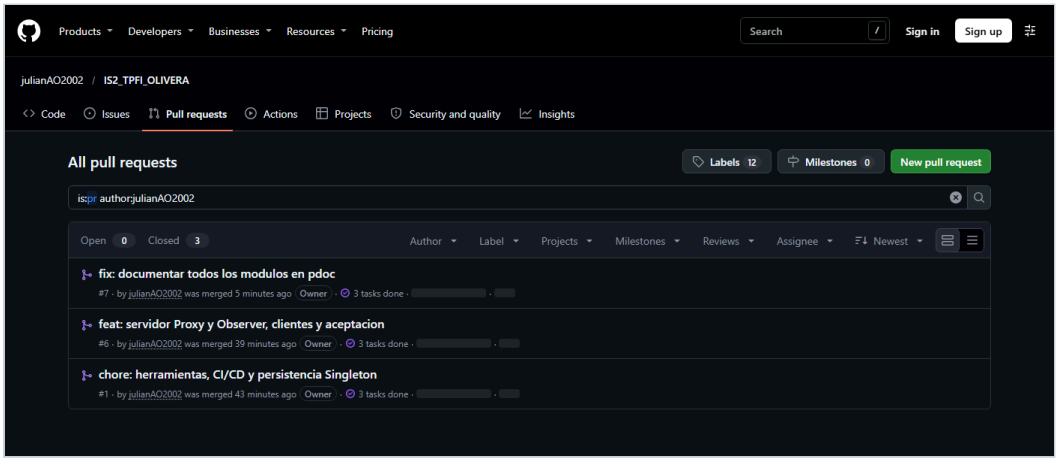


Figura 23. Condición: PRs del proyecto.

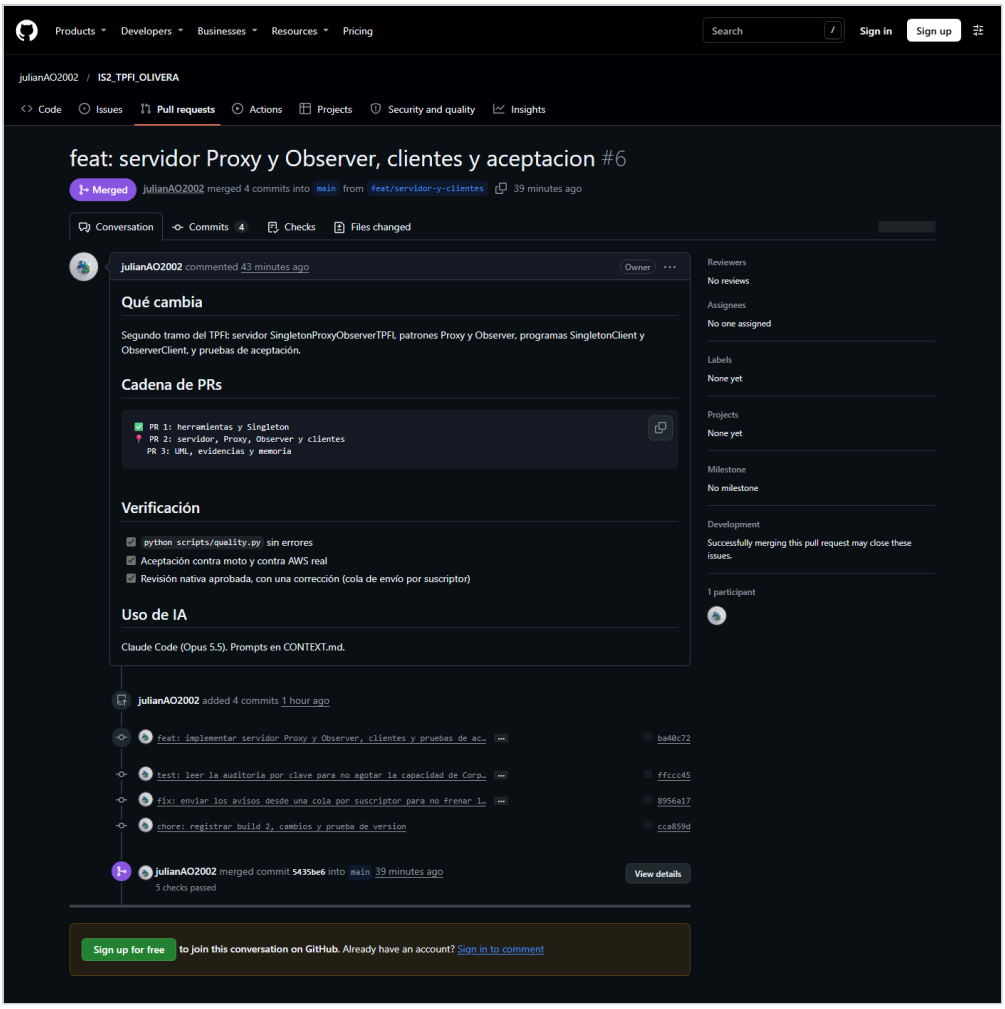


Figura 24. Condición: PR 6 integrado con sus 5 controles aprobados.

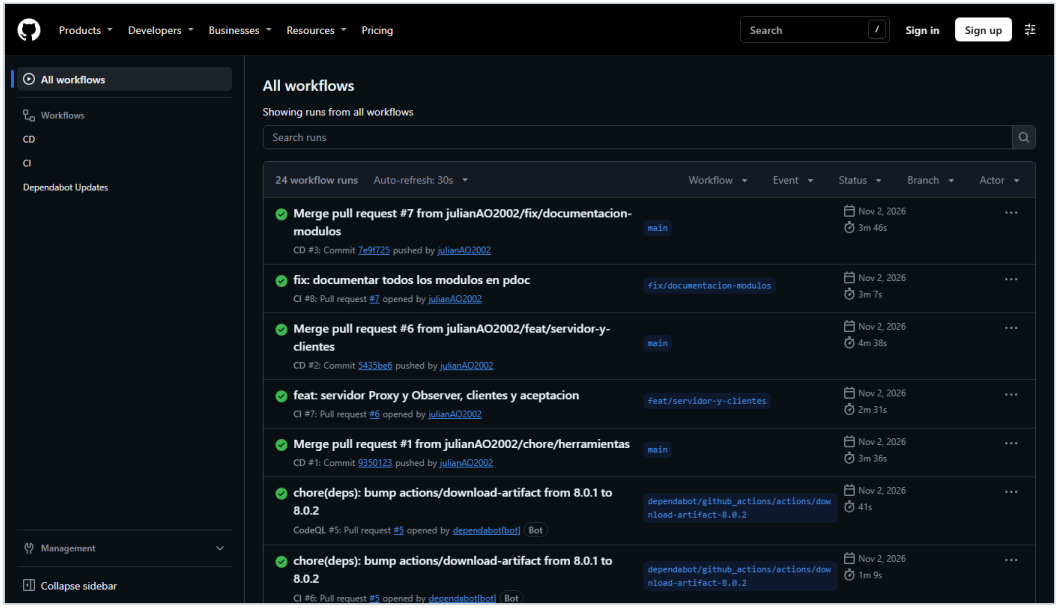


Figura 25. Condición: historial de ejecuciones de los workflows.

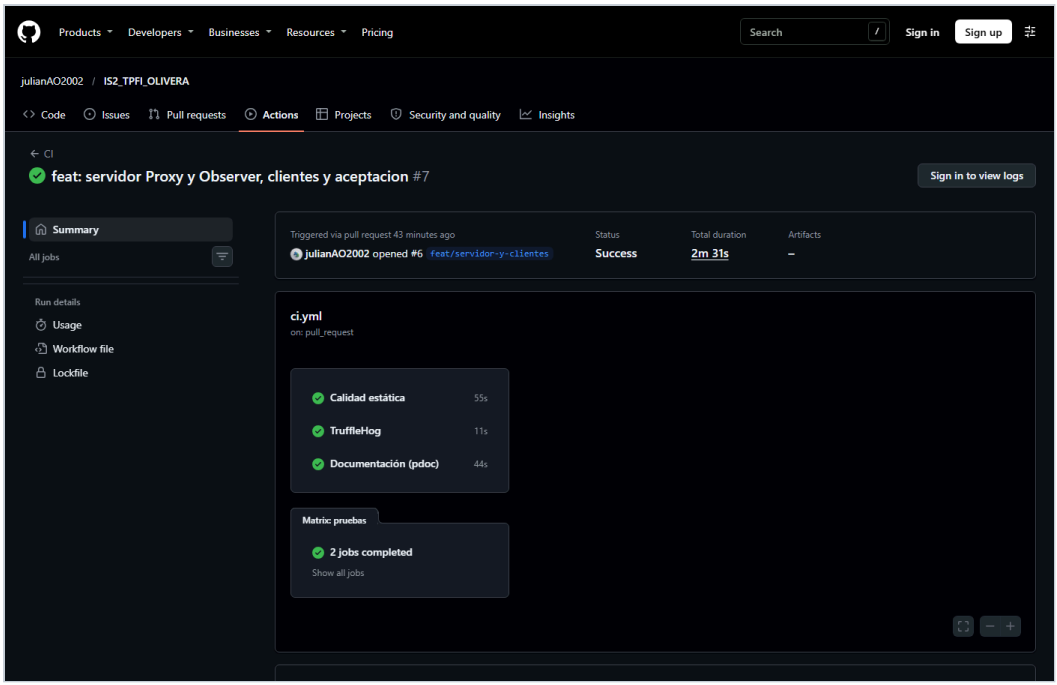


Figura 26. Condición: ejecución de CI del PR 6.

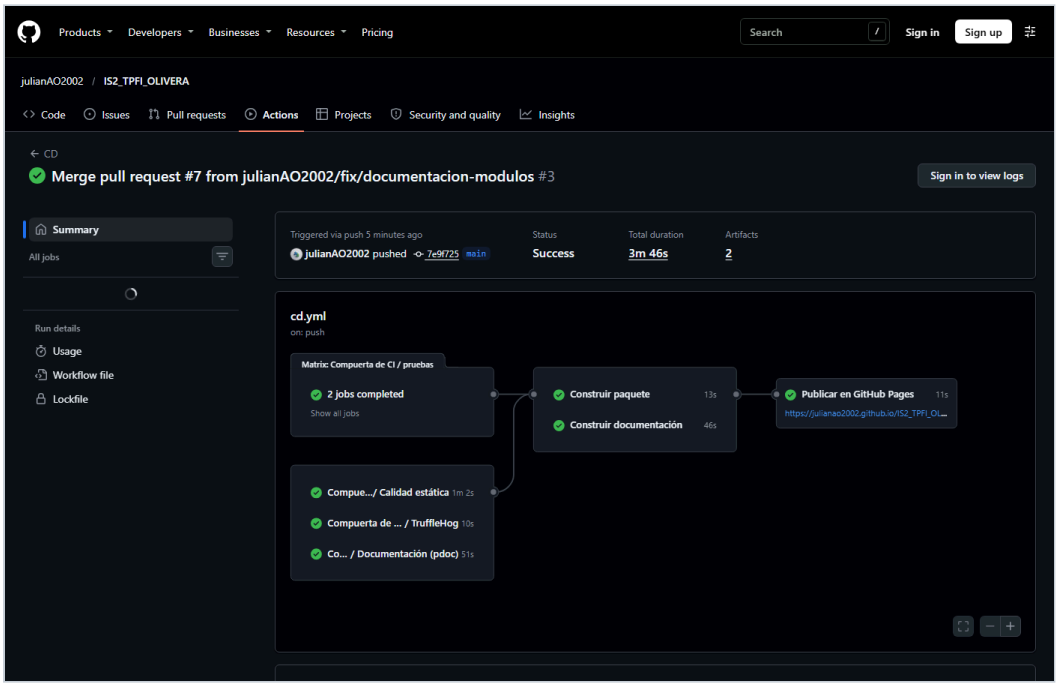


Figura 27. Condición: ejecución de CD: compuerta, paquete, documentación y publicación.

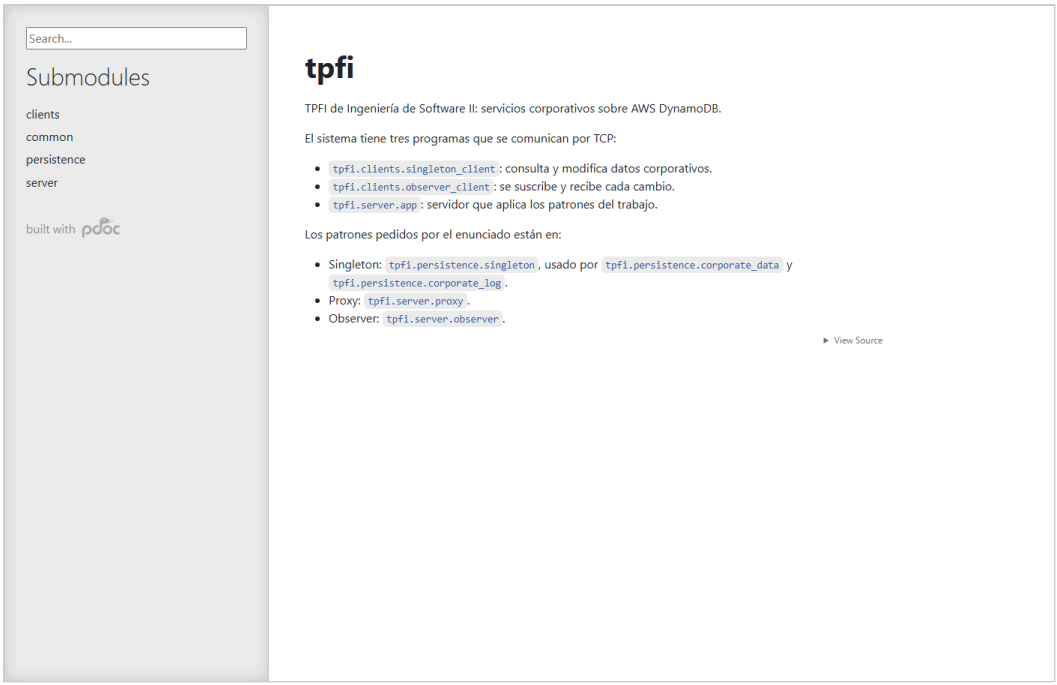


Figura 28. Condición: documentación publicada; actualizada al integrar el PR 7.

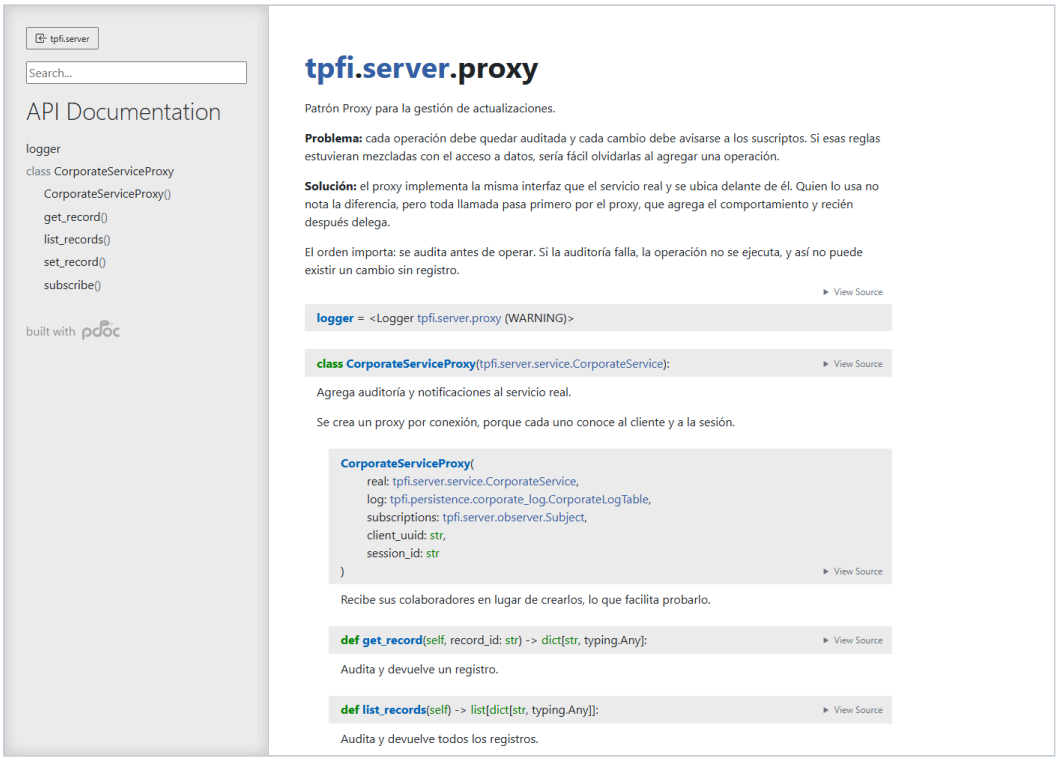


Figura 29. Condición: documentación del módulo proxy generada desde los docstrings.

Documentación publicada: https://julianao2002.github.io/IS2_TPFI_OLIVERA/tpfi.html

9. Cambios y defectos encontrados

Esta sección registra los problemas reales del desarrollo, porque muestran qué detectó cada técnica de verificación.

Defecto	Quién lo detectó	Corrección
`MessageReader` perdía un mensaje parcial si `recv` vencía su plazo, porque consumía la cabecera antes de tener el cuerpo.	Prueba unitaria escrita antes de implementar el cliente	Los bytes se consumen recién con el mensaje completo.
Un suscripto trabado demoraba la respuesta del set hasta 5 segundos por cliente. Con varios, el cliente podía recibir un timeout aunque el dato ya estuviera guardado.	Revisión de resiliencia (severidad crítica)	Cada `SocketObserver` tiene una cola acotada y un hilo de envío. Se agregaron 5 pruebas.
La primera corrida real falló en 8 de 32 casos con `ProvisionedThroughputExceededException`. El arnés de pruebas hacía un `Scan` de CorporateLog por prueba.	Corrida contra AWS real (moto no limita la capacidad)	Cada fila se lee por clave con `GetItem`. El sistema no tenía el defecto; lo tenía el arnés.
La documentación publicada no incluía los módulos: `__all__` limitaba a pdoc y el CI solo comprobaba que pdoc terminara bien.	Revisión de las capturas de esta memoria (la página del módulo proxy daba 404)	`__all__` incluye los subpaquetes, y `build_docs.py` falla si falta una página. Se agregaron 4 pruebas.
El esqueleto incluía `dependabot.yml`, que abrió 4 PRs sobre workflows ya descartados.	Revisión del estado de GitHub	Se cerraron con explicación; el archivo ya se había quitado en el PR 1.

> Impacto del tercer defecto: durante unos 5 minutos, el arnés consumió capacidad de lectura de una tabla compartida con otros alumnos, lo que pudo demorar sus lecturas. La corrida corregida tomó 68 segundos, contra 289 de la original.

Revisión independiente

El código se revisó con Gentle AI por tramos, porque el cambio completo superaba el tamaño admitido. El tramo del servidor pasó por las lentes de riesgo, resiliencia y fiabilidad, con una corrección. El tramo de protocolo y persistencia pasó por las lentes de riesgo y fiabilidad, sin correcciones. El arreglo de la documentación pasó por la lente de fiabilidad.

Registro de versiones

Build	Contenido
1	Esqueleto, herramientas, CI/CD, protocolo y persistencia Singleton
2	Servidor, Proxy, Observer, clientes y aceptación
3	Documentación pdoc con todos los módulos
4	UML, evidencias, memoria, README completo y pruebas de la generación de documentación

La demostración de las secciones 6.3 a 6.6 se hizo sobre el build 2, por eso esa versión aparece en las capturas. Los builds 3 y 4 no cambian el comportamiento de los programas. Los controles de calidad, la cobertura, la aceptación contra AWS y TruffleHog se repitieron sobre el build 4.

10. Limitaciones conocidas

La revisión dejó observaciones que no bloquean la entrega. No se resolvieron porque el enunciado no las pide, pero un sistema productivo tendría que atenderlas.

Limitación	Consecuencia	Cómo se resolvería
El servidor no autentica a los clientes	Quien alcance el puerto puede leer y modificar datos	Autenticación por token, o limitar la red de acceso
El UUID lo informa el cliente	La pista de auditoría se puede falsificar	Asociar la identidad a una credencial verificada
La comunicación no está cifrada	Los datos viajan legibles por la red	Usar TLS
Un hilo por conexión, sin tope	Muchas conexiones pueden agotar recursos	Usar un conjunto de hilos con límite
El apagado no espera a los hilos en curso	Una respuesta puede quedar cortada al cerrar	Apagado ordenado con espera
La cancelación manual se probó simulando la excepción	Ctrl+C no se ejercitó en una prueba automatizada	Prueba con señal real por proceso

Además, las evidencias públicas incluyen el identificador de CPU, derivado de la dirección MAC, porque el enunciado exige registrarlo.

11. Uso de inteligencia artificial

El enunciado permite usar herramientas de IA si se documentan los prompts. Se usó Claude Code con el modelo Claude Opus 5.5. La herramienta leyó el enunciado, propuso el diseño, escribió el código, las pruebas, los diagramas, los workflows y esta memoria, y ejecutó las verificaciones. Gentle AI aportó la revisión independiente. Los prompts están registrados textualmente en `CONTEXT.md`.

12. Conclusiones

- Los tres patrones separan bien las responsabilidades: agregar auditoría o notificaciones no requirió tocar el acceso a datos.
- Cada técnica encontró defectos distintos. La prueba unitaria detectó el error de lectura, el entorno real detectó el límite de capacidad y la revisión detectó el bloqueo por suscriptores trabados. Ninguna de ellas por sí sola los habría encontrado todos.
- Un entorno simulado es útil pero no alcanza: las pruebas pasaban 32 de 32 contra moto y 24 de 32 contra AWS en la primera corrida.
- La automatización de los controles permitió repetirlos en cada cambio, con el mismo criterio en la máquina local y en GitHub.

13. Anexo: ejecución del sistema

```
python -m venv .venv
.venv\Scripts\activate
python -m pip install -r requirements.txt
aws configure

python singletonproxyobserver.py -v
python observerclient.py -o=avisos.json
python singletonclient.py -i=examples/input_set.json
python singletonclient.py -i=examples/input_get.json -o=output.json
python singletonclient.py -i=examples/input_list.json

python scripts/quality.py
python -m pytest tests/acceptance -m acceptance
```

Repositorio: https://github.com/julianAO2002/IS2_TPFI_OLIVERA